Navi — Product & Engineering Brief

Version: 1.1 · **Decision baseline:** October 29, 2026 · **Audience:** Claude Code, building in a cloud environment
Repository: `https://github.com/adityaygit/navi`
Status: Product requirements locked; external credentials, actual environment access, and device-level verification are not yet established.

1. What Navi is

Navi is an AI-native personal operating system for organizing, planning, executing, and improving daily life. It begins as a private personal product, designed to support many users securely later; teenagers and young adults are its intended eventual audience. The platform and its assistant are both called **Navi**. "Life OS" describes the concept, not necessarily the brand.

Navi is **not** a to-do app with a chatbot attached. The AI can operate the same structured system a person sees in the dashboard: responsibilities, projects, goals, tasks, routines, schedules, progress, and memories. Conversational control and visual controls must stay synchronized.

Product loop: capture → understand → review/commit → plan → execute → reflect → improve.

V1 priority: a dead-basic, intuitive, reliable UI with working backend functionality. Claude chooses a simple, accessible, reusable design system. No need to await Refero, replicate a site, create mascots, or spend early effort on branding. The system must still be excellent on desktop and mobile; visual redesign comes later.

2. The first-run experience: Navi sets up your OS through conversation

This is a defining requirement, **not** a long form and **not** a task for the user to configure manually:

1. A user signs in using **Google or email**. Initial access is private/restricted; do not launch open public registration by accident.
2. Navi welcomes the user into a natural **text or voice** conversation. It learns about their current responsibilities, life areas, projects, goals, regular activities, fixed commitments, time availability, preferences, time zone, and quiet hours. Ask useful follow-ups rather than interrogating them with a huge questionnaire.
3. From that conversation, Navi **proposes an actual initial operating system**: workspaces, projects, goals, routines, confirmed calendar commitments, proposed tasks, and an initial week/day plan. Do not merely create generic topic labels or a starter checklist.
4. The user reviews and corrects the proposed setup in one clear flow. **Only reviewed/approved items become active commitments.** Preserve uncertain items in a review state. Show what was inferred versus explicitly stated.
5. Navi persists the setup and produces a populated dashboard and usable first plan. All data should be real and editable. Manual editing, document import, and external integrations are optional shortcuts, not onboarding prerequisites.

Onboarding must be resumable, skippable where practical, and repeatable through later conversations. It can gather app preferences conversationally instead of separate setup forms.

3. Core V1 experience

A. Hybrid interface: first-class UI and first-class AI

Provide **Today/Home, Navi Chat, Review Inbox, Planner, Tasks, Projects/Workspaces, Goals, Insights/Reviews, Memory, Activity/Approvals, and Settings**. These need not all be separate top-level navigation items; organize them intuitively. On desktop, Navi can be available contextually alongside other screens; on mobile, conversation and quick capture should be immediately accessible. Every displayed statistic and action must reflect real persisted data, not fabricated demos.

B. Shared text and real-time voice

Users can choose to type **or talk** and move between the two in the same conversation. Real-time two-way voice with natural turn-taking, interruption/barge-in, mic/mute/end controls, connection status, and usable transcripts is a **V1 requirement**. Both modes access the same context, memory, and permissioned backend actions. Save transcripts by default; **do not intentionally store raw audio** without explicit notice and choice. Voice starts only by user action, never via an always-listening microphone. Support recovery from network failures; do not falsely promise that iOS allows uninterrupted background audio. Voice requires an internet connection.

C. Capture and review inbox

Typing or speaking a messy brain dump creates **proposed** tasks, reminders, events, goals, notes, and questions. Preserve the raw words and distinguish source, confidence, and inference. The user can accept, edit, merge, reject, or defer them in the review inbox. **Casual conversation must not silently create committed tasks.** An explicit command to update an existing item can be acted on within the internal permission policy. Never silently discard captured information; offline captures must queue safely.

D. Tasks, projects, workspaces, and goals

Users need meaningful task and project management, not merely an inbox: tasks and subtasks, deadlines, priorities, estimates, statuses, recurrence when appropriate, project milestones, blockers, objectives, and relevant notes/links. Life areas such as School, Startups, Fitness, and Personal can have workspaces, but Navi must understand competing obligations across them. Support goal milestones and how daily work contributes to those goals. Keep it practical rather than building an enterprise PM suite.

E. Planner, execution, and recovery

Provide an interactive **day/week calendar**, fixed commitments versus flexible work blocks, deadline-aware priorities, estimated versus actual time, and focus-session/progress recording. Keep due dates distinct from scheduled work times. Navi can automatically reorganize **flexible internal** plans and explain notable changes, but must not quietly move external events, invent

availability, or mark scheduled work as completed. Include a fast **“my plan broke”** recovery flow: record what changed, retain unfinished work, and generate a realistic revised plan.

F. Memory and ongoing learning

Persist relevant user preferences and routines (**profile**), project decisions and context (**project**), meaningful events and outcomes (**episodic**), and cautiously inferred repeated patterns (**behavioral**). Remember between sessions. Users can inspect, correct, and delete memories; label origin, timestamps, and uncertainty. Treat AI inference differently from confirmed facts and allow later evidence to overturn it. Use past estimates and outcomes to improve future plans rather than making up statistics.

G. Strict but fair accountability and reviews

The user chose **strict** accountability. Navi should directly identify avoidable delays, repeated missed intentions, and unrealistic commitments, but never shame, manipulate, or attribute motives without evidence. Distinguish intentional rest, interruptions, lack of information, and inaccurate estimates. Track planned vs. actual progress and provide concrete recovery options. The evening review may offer an explainable completion rating; no fake A–F score based on assumptions. Provide weekly trend and improvement insights as enough evidence accumulates.

H. Proactive operations

Default **morning briefing at 06:00** and **evening review at 21:00**, both editable and based on the user's local time zone. **Quiet hours are user-configured** during onboarding, not predetermined. Persist generated briefings. Check upcoming deadlines and schedule conflicts, replan on relevant changes, and offer a weekly review. Background jobs must run server-side even when the app is closed, handle DST, deduplicate notifications, and avoid wasteful AI calls. Notifications should be meaningful and respect quiet hours.

I. Autonomy and approvals

Autonomous internally: Navi may organize data, generate plans, reprioritize flexible blocks, prepare drafts, and produce reviews. **Review-first** still applies to newly inferred commitments. For consequential actions affecting other people or external systems (sending messages, modifying connected calendars, submitting work, spending money), display exactly what will happen and obtain explicit approval. Changes to real deadlines, cancellation of important commitments, and destructive data changes also need approval. Log outcomes and enable reversal where feasible. An AI tool response must never claim an external action completed without actual confirmation.

4. Accounts, iPhone PWA, privacy, and operations

- **Authentication:** Google sign-in **and** email login via a secure auth service. Private/invite-restricted launch; account ownership and database isolation designed for future multi-user support. Do not substitute a fake login in the finished V1.
- **Web + iPhone:** A responsive website that can be **installed from Safari on iPhone** as a standalone PWA. On mobile prioritize Today, Navi, capture, planner, inbox, and pending approvals. Support safe-area insets, touch targets, keyboard behavior, and preserved drafts.

- **Push notifications are V1:** iPhone Home Screen web push with user permission, settings, delivery diagnostics, and relevant deep links. Physical installed-iPhone verification is required for sign-off; if unavailable, identify that as **unverified**, not completed. Follow platform/browser limitations honestly.
- **Basic offline is V1:** Cache previously synced essentials, allow offline note/task capture into a visible queue, and sync on reconnect with safe conflict handling. No requirement for offline AI or advanced offline editing. Never cache private user data in a shared/public cache.
- **Privacy and safety:** Use server-side secrets, row-level/user-scoped database access, minimal retention, export/delete controls, visible microphone state, honest provider retention disclosures, prompt-injection resistance for imported content, and actionable failure states. Avoid invasive tracking in the personal V1. Future public teen release requires separate age, privacy, consent, and provider-terms review.
- **AI testing budget: $2/day**, chosen by the user for initial testing, not a permanent subscription limit. Track approximate real usage/cost for **all providers, including voice**, warn, and conservatively stop new testing calls at the cap unless explicitly overridden. Expose uncertainty where provider cost reporting is delayed. Also recommend provider-side billing alerts/limits; an app-side guard alone cannot absolutely cap charges. Configure budget reset behavior sensibly and document it.

5. Technical preferences and AI runtime

Preferred technology choices are **Next.js + TypeScript + Tailwind/shadcn-style simple components, Supabase (Postgres/Auth/Storage), Vercel, Trigger.dev, and a private GitHub repository**. This is a starting direction, not a demand to micromanage every library. Claude may choose justified simpler alternatives for the existing repo or verified constraints. Use secure multi-user data ownership, versioned migrations, and durable, retry-safe background tasks.

App runtime model roles (distinct from whatever model Claude Code uses for building):

Role	Selected starting model	Verified official API model ID
Low-cost everyday capture, short chat, summaries	Claude Haiku 5.5	`claude-haiku-5-5`
Complex planning/reasoning	Claude Sonnet 5.5	`claude-sonnet-5-5`
Real-time two-way voice	Gemini 3.8 Live	`gemini-3.8-live`

These identifiers are documented as of **October 29, 2026**, but Claude must still verify **account availability, exact SDK behavior, streaming, provider pricing, and terms in its actual environment**. Keep provider selection configurable, support a usable alternate/fallback, and benchmark representative Navi operations for quality/latency/cost. Use deterministic application code instead of model calls for simple calculations or database operations. Do not hardcode credentials or provider model IDs all over the app. The primary role for voice must satisfy the shared-history and shared-tools requirements, not behave like a disconnected second assistant.

Official technical references to recheck in Claude's environment:

- Anthropic model IDs and current status: https://platform.claude.com/docs/en/models/overview

- Google Gemini 3.8 Live model: https://ai.google.dev/gemini-api/docs/models/gemini-3.8-live
- Google Live API: https://ai.google.dev/gemini-api/docs/live-api
- Next.js PWA guide: https://nextjs.org/docs/app/guides/progressive-web-apps

Integrations: Build a clean extension point for Google Calendar, Gmail, GitHub, Drive, and future authorized school platforms, but **do not treat all these integrations as V1 completion requirements**. The internal planner/calendar is V1. External services should be connected only through documented authorizations and scoped permissions.

6. How Claude should execute this build

Cloud environment, real repo: `https://github.com/adityaygit/navi`. The user has created it, but **read/write or push access has not been verified**. Start by inspecting the repo and testing an authorized harmless development-branch push early, so a Fairylearn-style late 403 does not trap dozens of commits. Don't assume Claude Cloud can control the user's laptop, Edge browser, logged-in Google session, or personal accounts.

Claude is the **orchestrator**: research the environment; set up everything already authorized through available CLIs/integrations; choose sensible implementation details; delegate independent tasks to ordinary subagents as useful; integrate and test. Favor working vertical slices over decorative screens. If an external service requires credentials or account consent, log the blocker and make progress on independent tasks using isolated dev substitutes, but never misrepresent a stub as a production integration. No surprise purchases, exposed secrets, destructive production changes, or external communications. The user's development permission settings do not authorize those consequences.

Maintain **Build Intelligence** as project-local, durable state: full V1 roadmap; verified vs. merely implemented milestones; important findings, decisions and blockers; tests; Git commits; and an exact next action. Keep this lean, not a second application. Checkpoint regularly. On **"resume"**, inspect Git and persisted project state, reconcile differences, and continue rather than repeat planning. Cloud sessions may stop at usage limits or require manual restart; do not promise automatic renewal. Persist verified progress remotely when push access permits and keep a recoverable local/export checkpoint otherwise.

7. V1 acceptance gates (test, don't merely claim)

1. Sign in with Google **and** email in a configured environment; enforce user-to-user isolation and private access.
2. Complete conversational **text or voice** onboarding; approve/edit the proposed **workspaces, projects, routines, goals, commitments, and first plan**; see actual saved results immediately.
3. Capture messy text or voice input; preserve source wording; review/convert/reject proposals; survive reload and synchronize all interfaces.
4. Use real Today, Planner, Tasks, Projects/Goals, Insights, Memory, and Activity/Approvals views; relevant controls work, with honest empty states.
5. Manage fixed commitments, flexible time blocks, tasks/focus sessions, and recovery/replanning; never silently lose unfinished work.

6. View/correct/delete persistent memory with provenance; retrieve relevant project context after a new session.
7. Receive saved 06:00 and 21:00 local scheduled briefings, plus meaningful deadline reminders, with timezone/DST and configurable quiet-hour behavior.
8. Observe strict but fair accountability grounded in actual events, not fabricated progress.
9. Verify internal autonomy and the external approval boundary, including failure and duplicate-action protection.
10. Talk to Navi in **working real-time voice**, interrupt it, and return to text while preserving conversation and authorized tools.
11. Install PWA from iPhone, receive opted-in push, and capture offline/sync correctly. Track any real-device step not yet verified as an outstanding launch gate.
12. Record provider usage and enforce a conservative $2/day **testing** guard with warnings and transparent behavior.
13. Verify backend persistence, secret protection, access controls, automated checks, mobile usability, and a deployed private preview where deployment credentials allow.
14. Verify a new Claude session can **resume** from the repository and development checkpoint without re-explaining the product.

Incomplete integrations, unconfigured credentials, or untested phone-only behaviors must be reported honestly. A functioning demo is not the same as a fully verified V1. Finish as much as possible autonomously, and give the user only the specific setup/approval steps that truly need them.